



Proyecto Intermodular: Diseño

Kajas: Gestión Inteligente de Almacenamiento

Alumno: David Lara Tarazaga
Grupo: 2ºDAW

1 de Diciembre de 2025

Índice

1. Arquitectura del Sistema	3
1.1. Definición de la Arquitectura	3
1.2. Diagrama de Arquitectura Técnica	3
1.3. Justificación de las Tecnologías	4
2. Diseño de la Base de Datos	5
2.1. Modelo Entidad-Relación (E/R)	5
2.2. Descripción de Tablas y Relaciones	6
2.3. Tecnología de Persistencia	6
3. Diseño de Interfaz y Diagramas	7
3.1. Flujo de Funcionamiento (Diagrama de Flujo)	7
3.2. Diagrama de Clases (UML)	8
3.3. Prototipos de Interfaz (Wireframes/Mockups)	9
3.3.1. Pantalla de Inicio y Dashboard	9

1. Arquitectura del Sistema

1.1. Definición de la Arquitectura

Para el desarrollo de la aplicación **Kajas**, se ha seleccionado una arquitectura basada en el modelo **Cliente-Servidor** utilizando una estructura de **API REST**. Esta arquitectura desacopla completamente el frontend del backend, permitiendo escalabilidad y mantenimiento independiente.

El sistema sigue el patrón de diseño **MVC (Modelo-Vista-Controlador)** en el lado del servidor para organizar la lógica de negocio.

- **Cliente (Frontend):** Aplicación SPA (Single Page Application) desarrollada en React. Se encarga de la interfaz, la experiencia de usuario y el escaneo de QR.
- **Servidor (Backend):** API RESTful desarrollada en Node.js con Express. Gestiona las peticiones, la autenticación y la lógica de negocio.
- **Base de Datos:** Sistema relacional PostgreSQL para la persistencia de datos.

1.2. Diagrama de Arquitectura Técnica

A continuación, se representa gráficamente la comunicación entre los componentes del sistema:

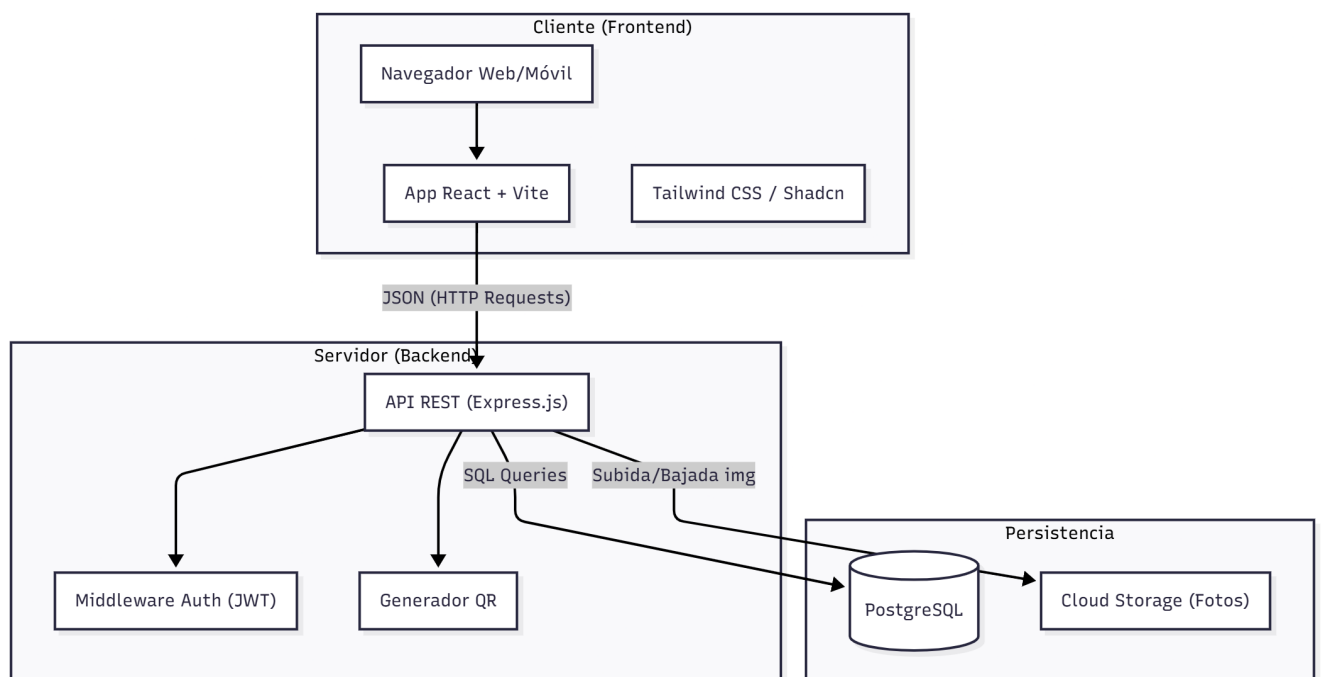


Figura 1: Diagrama de Arquitectura de Kajas (Stack MERN/PERN)

1.3. Justificación de las Tecnologías

- **Frontend (Vite + React + Tailwind):** Se elige React por su ecosistema y capacidad de gestión de estado, vital para una aplicación interactiva. Vite proporciona un entorno de desarrollo rápido. Tailwind y Shadcn/ui garantizan una interfaz cozy y responsive con accesibilidad nativa.
- **Backend (Node.js + Express):** Permite utilizar JavaScript en todo el stack (Full Stack JS), simplificando el desarrollo. Su naturaleza asíncrona es ideal para manejar múltiples peticiones I/O simultáneas.
- **Persistencia (PostgreSQL):** Al tratarse de datos altamente estructurados y relacionales (Un Usuario -> Muchas Ubicaciones -> Muchas Cajas -> Muchos Objetos), una base de datos SQL es superior a NoSQL para garantizar la integridad referencial.

2. Diseño de la Base de Datos

2.1. Modelo Entidad-Relación (E/R)

La base de datos se ha diseñado siguiendo las reglas de normalización (hasta la 3FN) para evitar redundancias. Las entidades principales son Usuarios, Ubicaciones, Cajas y Objetos.

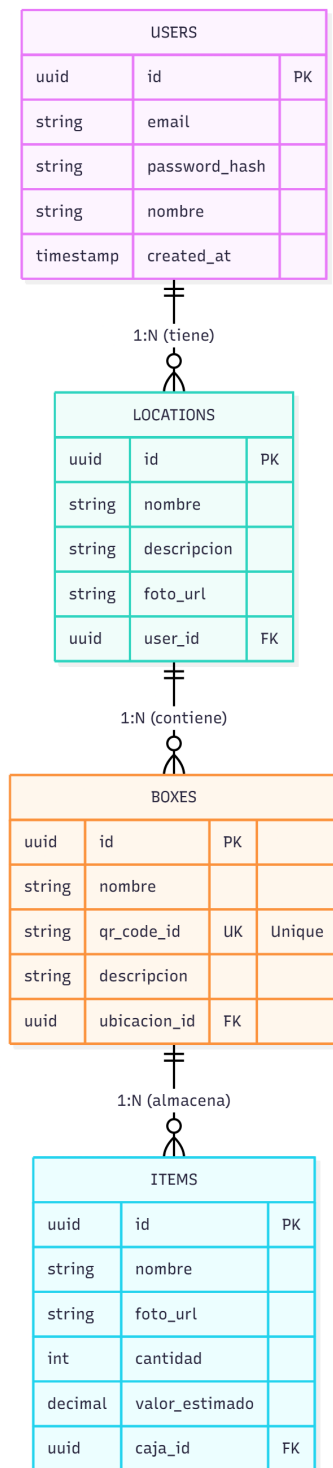


Figura 2: Diagrama Entidad-Relación de Kajas

2.2. Descripción de Tablas y Relaciones

1. **Usuarios:** Almacena las credenciales y datos de perfil.
2. **Ubicaciones:** Representa espacios físicos (ej. Trastero, Oficina). Tiene una relación 1:N con Usuarios (un usuario tiene muchas ubicaciones).
3. **Cajas:** Representa el contenedor físico. Tiene una relación 1:N con Ubicaciones. Incluye el campo único `qr_code_id`.
4. **Objetos:** Los ítems individuales. Relación 1:N con Cajas.

2.3. Tecnología de Persistencia

Se utilizará **PostgreSQL**. Se justifica su elección frente a MySQL por su mejor manejo de tipos de datos complejos y robustez en integridad de datos, y frente a MongoDB porque la estructura de datos de Kajas es fija y relacional, no documental.

3. Diseño de Interfaz y Diagramas

3.1. Flujo de Funcionamiento (Diagrama de Flujo)

El siguiente diagrama muestra el flujo principal del usuario: desde el login, creación de una caja, hasta la generación y escaneo del QR.

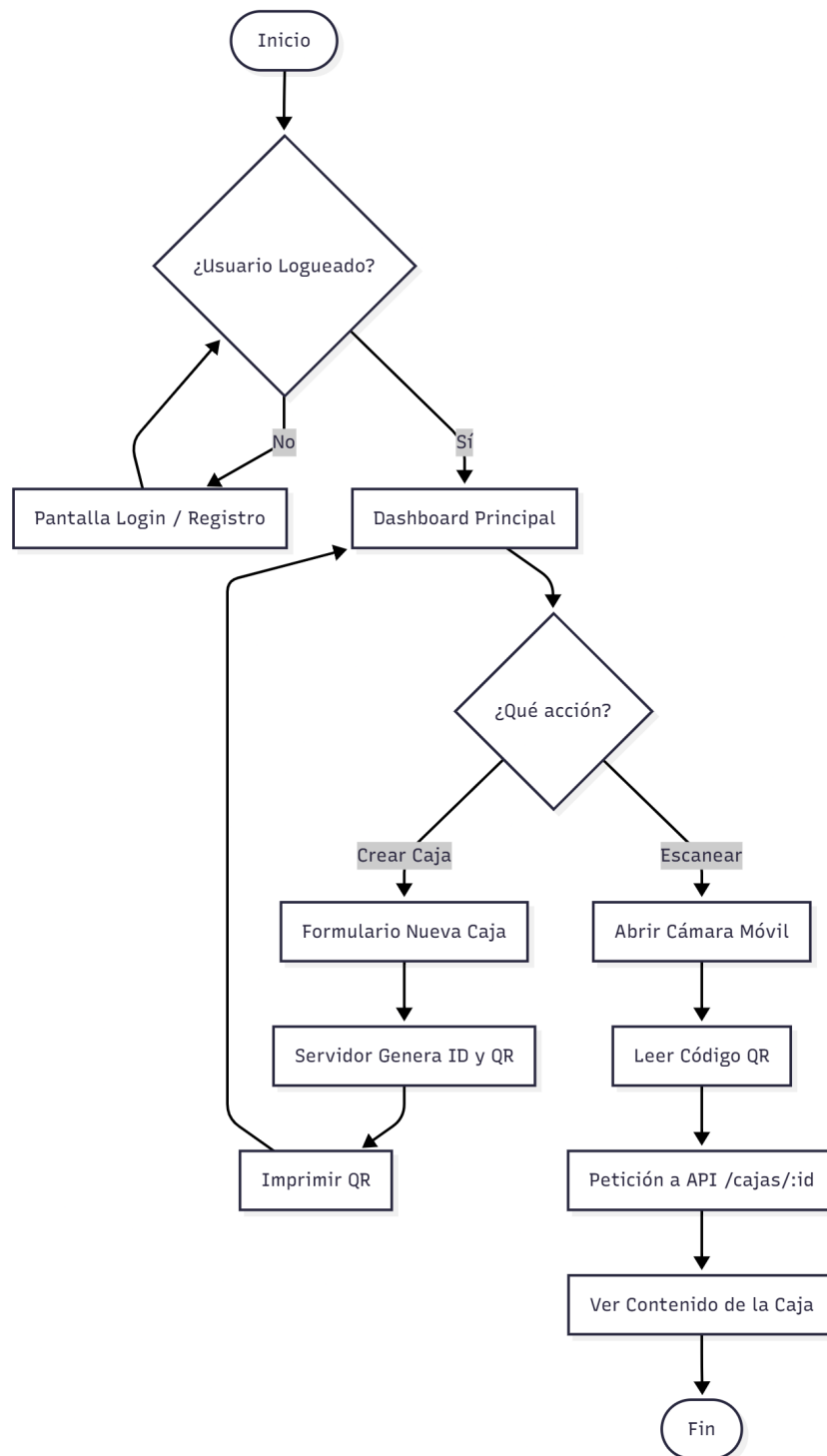


Figura 3: Flujo de usuario principal: Crear Caja y Escanear

3.2. Diagrama de Clases (UML)

Representación de la estructura de clases del Backend (Controladores y Modelos) y sus relaciones.

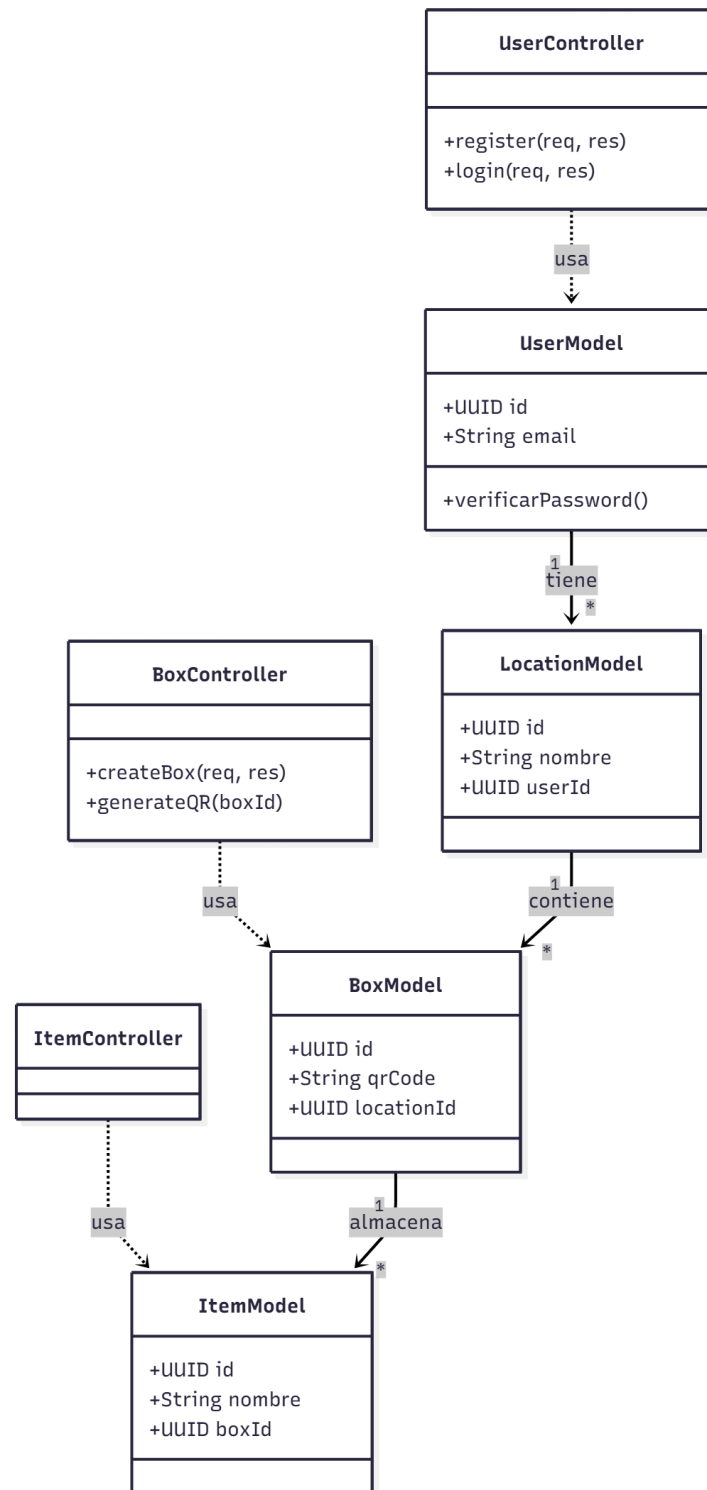


Figura 4: Diagrama de Clases UML

3. Diseño de UI

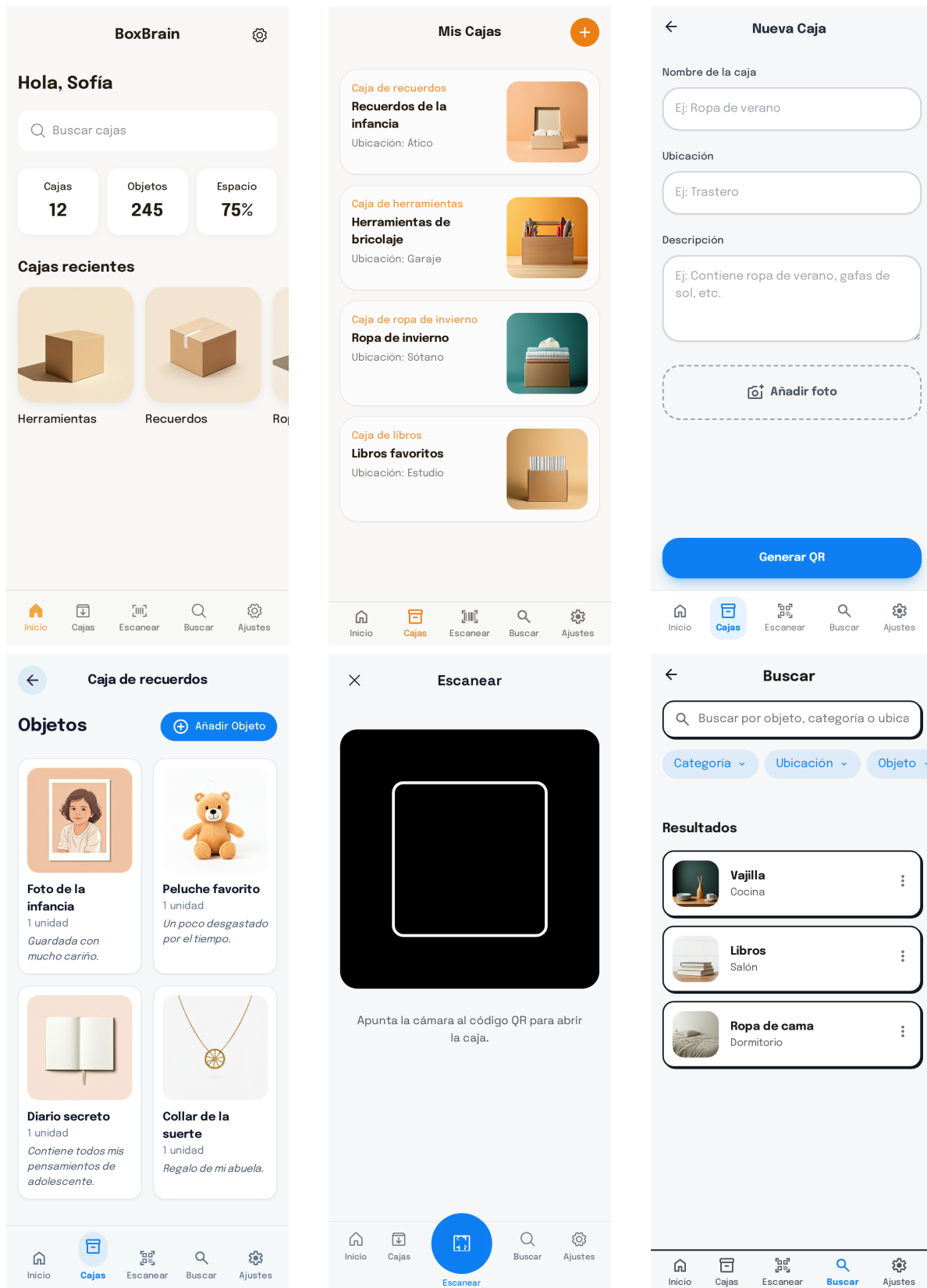


Figura 1: Ejemplos del diseño de la interfaz de usuario de Kajas.